

Tarea 1 2026

Tópicos en Grandes Volúmenes de Datos

Count-Min Sketch y CountSketch en ventanas deslizantes para detección de ataques

Cecilia Hernández y Álvaro Guzmán

Fecha de inicio: Viernes 11 de septiembre de 2026.

Fecha de entrega: Jueves 1 de octubre de 2026, 23:59 hrs.

Grupos: 3 estudiantes.

1. Objetivo

El objetivo de esta tarea es estudiar cómo dos sketches de frecuencia, **Count-Min Sketch (CMS)** y **CountSketch (CS)**, pueden mantenerse sobre una **ventana deslizante** y utilizarse para detectar cambios anómalos en tráfico de red.

La idea algorítmica central es aprovechar que ambos sketches son **lineales**. En lugar de reconstruir el sketch de una ventana cada vez que ésta avanza, el flujo se divide en subventanas temporales. Cada subventana tiene su propio sketch y el sketch de la ventana activa se obtiene sumando los sketches vigentes. Cuando la ventana avanza, el sketch que expira se resta y el nuevo se incorpora.

La tarea se concentra únicamente en dos ataques sintéticos:

- **DDoS**: muchas fuentes envían tráfico hacia una misma víctima. Se analiza frecuencia por **IP de destino**.
- **Scan**: una fuente contacta muchos destinos. Se analiza frecuencia por **IP de origen**.

No se evaluará el problema general de enumerar todas las claves candidatas desde un sketch. El generador de ataques entrega la clave involucrada en un archivo JSON, y esa clave se utilizará para estudiar la calidad del estimador y la latencia con que el ataque se vuelve detectable. El objetivo es aislar el comportamiento de los sketches y del mecanismo de ventana deslizante.

2. Datos y herramientas entregadas

Se utilizará una traza del [MAWI Working Group Traffic Archive](https://mawi.wide.ad.jp/mawi/samplepoint-F/2018/201812031400.pcap.gz), punto de captura **samplepoint-F**. Se recomienda utilizar una traza entre 2016 y 2019. Como referencia puede utilizarse:

<https://mawi.wide.ad.jp/mawi/samplepoint-F/2018/201812031400.pcap.gz>

El material entregado incluye:

- `pcap2bin.cpp`: convierte la traza pcap a registros binarios de 24 bytes.
- `exact_hh.cpp`: calcula frecuencias exactas por ventana y sirve como referencia.
- `inject_attack.py`: inyecta ataques sintéticos. En esta tarea sólo se usarán `ddos` y `scan`.
- `README.md` y `Makefile`: instrucciones y compilación de las herramientas entregadas.

Ejemplo de conversión:

```
g++ -O2 -march=native -o pcap2bin pcap2bin.cpp
zcat 201812031400.pcap.gz | ./pcap2bin > traza.bin
```

Ejemplos de ataques:

```
./inject_attack.py ddos --base traza.bin --out traza_ddos.bin \
    --gt gt_ddos.json --start 300 --duration 30 \
    --pps 10000 --sources 4000

./inject_attack.py scan --base traza.bin --out traza_scan.bin \
    --gt gt_scan.json --start 300 --duration 30 \
    --pps 8000 --dst-count 60000
```

Cada paquete inyectado queda además marcado en el campo `flags`, por lo que existe un ground truth exacto para verificar los experimentos.

La intensidad `--pps` está elegida para que el error de los sketches sea visible: si el ataque es demasiado fuerte, su frecuencia supera por varios órdenes de magnitud al ruido de las colisiones y todas las configuraciones de w dan prácticamente el mismo resultado. Si al graficar observan que las curvas estimada y exacta se superponen en los tres anchos, bajen `--pps` en vez de concluir que no hay nada que comparar.

La IP que debe consultarse está indicada en el JSON: `ataque.victima` para DDoS y `ataque.atacante` para scan. Por ejemplo, una consulta exacta puede ejecutarse como:

```
./exact_hh traza_ddos.bin --key dst -W 60 --delta 10 --phi 0.01 \
    --query <IP_victima> --out-query exact_ddos.csv

./exact_hh traza_scan.bin --key src -W 60 --delta 10 --phi 0.01 \
    --query <IP_atacante> --out-query exact_scan.csv
```

La salida de `--out-query` contiene, para cada ventana, la frecuencia exacta, la decisión de heavy hitter y el cambio exacto $\Delta f_j(x)$.

3. Heavy hitters y criterio de detección

Sea N_j el número total de paquetes de la ventana activa $\mathcal{W}_j$ y sea $f_j(x)$ la frecuencia exacta de una clave x en dicha ventana. Para un parámetro $\phi \in (0, 1)$, una clave x se denomina **heavy hitter** en la ventana j si

$$f_j(x) \geq T_j, \quad T_j = \lceil \phi N_j \rceil.$$

En esta tarea se utilizará $\phi = 0,01$. Por lo tanto, una clave es heavy hitter si representa al menos aproximadamente el 1 % de los paquetes presentes en la ventana de 60 segundos.

El programa `exact_hh.cpp` se utilizará como **referencia exacta**. Para cada ventana calcula $f_j(x)$ mediante conteo explícito y permite decidir

$$\text{HH}_j^{\text{exacto}}(x) = \begin{cases} 1, & f_j(x) \geq T_j, \\ 0, & f_j(x) < T_j. \end{cases}$$

CMS y CountSketch deben realizar la misma decisión, pero utilizando la frecuencia estimada $\hat{f}_j(x)$ obtenida desde el sketch agregado de la ventana:

$$\text{HH}_j^{\text{sketch}}(x) = \begin{cases} 1, & \hat{f}_j(x) \geq T_j, \\ 0, & \hat{f}_j(x) < T_j. \end{cases}$$

El valor N_j debe mantenerse de manera exacta mediante el anillo de contadores escalares descrito más adelante; no es necesario estimarlo mediante un sketch.

Cómo se modelan los dos ataques

En ambos casos el archivo JSON generado por `inject_attack.py` entrega la clave asociada al ataque. No se solicita descubrir esa clave recorriendo todas las posibles IPs.

- **DDoS:** la clave es la **IP de destino** de la víctima. Para cada ventana se considera

$$f_j(x) = \#\{\text{paquetes de } \mathcal{W}_j \text{ con dstIP} = x\}.$$

El DDoS se considera detectado en la primera ventana en que la IP víctima es clasificada como heavy hitter.

- **Scan:** la clave es la **IP de origen** del atacante. Para cada ventana se considera

$$f_j(x) = \#\{\text{paquetes de } \mathcal{W}_j \text{ con srcIP} = x\}.$$

El scan se considera detectado en la primera ventana en que la IP atacante es clasificada como heavy hitter.

Este criterio permite detectar un **scan de alta tasa**. No pretende resolver el problema general de detectar scans lentos, para lo cual sería más apropiado estimar, por ejemplo, el número de destinos distintos contactados por cada origen.

Ejemplo pequeño

Suponga una ventana con $N_j = 10\,000$ paquetes y $\phi = 0,01$. Entonces

$$T_j = \lceil 0,01 \times 10\,000 \rceil = 100.$$

Si el JSON indica que la víctima de un DDoS es la IP x y `exact_hh` obtiene

$$f_j(x) = 135,$$

entonces la referencia exacta indica ataque detectado porque $135 \geq 100$.

Si para la misma ventana CMS estima $\hat{f}_j(x) = 142$ y CountSketch estima $\hat{f}_j(x) = 128$, ambos sketches también detectan el ataque. En cambio, si CountSketch estimara $\hat{f}_j(x) = 94$, produciría un falso negativo respecto de la referencia exacta. De forma análoga, una estimación de CMS superior a 100 cuando $f_j(x) < 100$ produciría un falso positivo.

La misma lógica se aplica al scan, cambiando únicamente la clave observada: IP de destino para DDoS e IP de origen para scan.

4. Ventana deslizante mediante sketches lineales

Sea $f_j(x)$ la frecuencia de la clave x en la ventana

$$\mathcal{W}_j = (\tau_j - W, \tau_j],$$

donde W es el ancho de la ventana y las evaluaciones se realizan cada p segundos. En las herramientas entregadas, p corresponde a la opción `--delta`.

En esta tarea se utilizará una única configuración temporal:

$$W = 60 \text{ s}, \quad p = 10 \text{ s}, \quad m = W/p = 6.$$

Por lo tanto, cada ventana está formada exactamente por las últimas seis subventanas de 10 segundos. Las evaluaciones deben estar alineadas con los límites de estas subventanas.

Sea S_q el sketch construido con los paquetes de la subventana q . Por linealidad, el sketch de la ventana activa es

$$A_j = \sum_{q=j-m+1}^j S_q.$$

Al avanzar una subventana no es necesario volver a procesar los paquetes de los últimos 60 segundos:

$$A_{j+1} = A_j - S_{j-m+1} + S_{j+1}.$$

Esta actualización incremental es obligatoria. Se debe mantener un anillo de $m = 6$ sub-sketches y un sketch agregado A . Al expirar una subventana, sus contadores se restan de A , el sub-sketch se limpia y se reutiliza.

Alineación exacta de las subventanas. Esta parte debe implementarse con cuidado, porque un desajuste de una subventana produce un error que **no se nota en la mayoría de las ventanas** y aparece solo cuando el ataque empieza a salir de la ventana, que es justamente el tramo sobre el que se promedia el error. Fijen la convención siguiente:

- La subventana q cubre el intervalo $(t_0 + (q - 1)p, t_0 + qp]$, donde t_0 es la marca de tiempo del **primer paquete de la traza**. Es decir, un paquete cuya marca de tiempo cae exactamente en un borde pertenece a la subventana que **termina** en ese borde, no a la que empieza. La numeración empieza en $q = 1$, de modo que la ranura del anillo es $(q - 1) \bmod m$.
- Los instantes de evaluación son $\tau_j = t_0 + W + jp$. En τ_j el agregado debe contener exactamente las subventanas $q_\tau - m + 1, \dots, q_\tau$, con $q_\tau = (\tau_j - t_0)/p$.
- Dentro de cada evaluación, **primero** se expira la subventana más antigua y se resta de A , y **después** se cargan los paquetes de la subventana nueva.
- El anillo requiere precarga: en la primera evaluación $\tau_0 = t_0 + W$ las seis subventanas ya deben estar llenas con los paquetes de $(t_0, t_0 + W]$, cada uno en la subventana que le corresponde.

Ejemplo: las dos primeras ventanas

Tomando t_0 como origen, la subventana q cubre $(t_0 + (q - 1)p, t_0 + qp]$, es decir que S_q termina en $t_0 + qp$. Como la numeración empieza en $q = 1$, la ranura del anillo que le corresponde es

$$\text{ranura}(q) = (q - 1) \text{ mód } m.$$

Con $W = 60$ s, $p = 10$ s y $m = 6$:

Subventana	Intervalo	Ranura
S_1	$(t_0, t_0 + 10]$	0
S_2	$(t_0 + 10, t_0 + 20]$	1
S_3	$(t_0 + 20, t_0 + 30]$	2
S_4	$(t_0 + 30, t_0 + 40]$	3
S_5	$(t_0 + 40, t_0 + 50]$	4
S_6	$(t_0 + 50, t_0 + 60]$	5
S_7	$(t_0 + 60, t_0 + 70]$	0

Cuadro 1: Las siete primeras subventanas. Con $m = 6$ ranuras, S_7 reutiliza la de S_1 . En general la subventana que entra y la que expira comparten ranura, porque $(q - 1) \text{ mód } m = (q - m - 1) \text{ mód } m$.

La primera evaluación ocurre en $\tau_0 = t_0 + W$ y la segunda en $\tau_1 = \tau_0 + p$. El contenido del anillo en cada una es:

Evaluación	ranura del anillo						Ventana activa	Agregado
	0	1	2	3	4	5		
$\tau_0 = t_0 + 60$	S_1	S_2	S_3	S_4	S_5	S_6	$(t_0, t_0 + 60]$	$A_0 = \sum_{q=1}^6 S_q$
$\tau_1 = t_0 + 70$	S_7	S_2	S_3	S_4	S_5	S_6	$(t_0 + 10, t_0 + 70]$	$A_1 = A_0 - S_1 + S_7$

Cuadro 2: Contenido del anillo en las dos primeras evaluaciones. Entre una y otra cambia una sola ranura, la 0: S_1 expira, se resta de A , se limpia y esa misma ranura pasa a acumular S_7 . Las otras cinco no se tocan.

De aquí se desprenden tres cosas que conviene tener claras antes de programar:

- **El anillo contiene siempre exactamente $m = 6$ sub-sketches.** Al avanzar la ventana no se crea uno nuevo: se reutiliza la ranura del que expira. Con el sketch agregado A , son **siete** arreglos de $d \times w$ por cada tipo de sketch, y ese número no cambia durante toda la ejecución.
- **La actualización toca $2dw$ contadores,** independientemente de cuántos paquetes contenga la ventana de 60 segundos. Ése es el efecto de la linealidad que esta tarea busca mostrar.
- **En τ_0 el anillo ya debe estar lleno.** Los paquetes de $(t_0, t_0 + 60]$ se reparten entre las seis subventanas según el intervalo al que pertenecen; no se cargan todos en una sola ranura.

Noten además que, como $p = W/m$, dos ventanas consecutivas comparten cinco de las seis subventanas. Por lo tanto

$$\Delta A_j = A_j - A_{j-1} = S_{q_j} - S_{q_j-m},$$

es decir, la diferencia entre el sub-sketch que entra y el que sale. El vector de diferencias puede calcularse así, antes de limpiar la ranura que expira, sin almacenar una copia de A_{j-1} .

Autoverificación obligatoria. El anillo de seis contadores escalares entrega un N_j que debe coincidir **exactamente**, ventana por ventana, con la columna $\mathbb{N}$ que produce `exact_hh`. Implementen esa comparación antes de medir cualquier otra cosa: una sola ventana con N_j distinto indica que el anillo está desalineado, y en ese caso todos los resultados posteriores son inválidos. Es la verificación más barata y más útil de toda la tarea.

Importante. En Count-Min Sketch la resta se utiliza aquí únicamente para retirar de A un bloque que efectivamente formaba parte de la ventana. El sketch agregado resultante sigue representando un vector de frecuencias no negativas, por lo que el estimador habitual de CMS continúa siendo válido.

5. Sketches a implementar

5.1. Count-Min Sketch

CMS utiliza una matriz de $d \times w$ contadores. Para una actualización (x, c) :

$$C[j, h_j(x)] \leftarrow C[j, h_j(x)] + c, \quad j = 1, \dots, d.$$

La estimación es

$$\hat{f}(x) = \min_j C[j, h_j(x)].$$

CMS sobreestima la frecuencia debido a colisiones. En esta tarea todas las actualizaciones corresponden a conteo de paquetes, por lo que $c = 1$.

5.2. CountSketch

CountSketch utiliza, para cada fila, una función hash de posición h_j y una función de signo $s_j(x) \in \{-1, +1\}$:

$$C[j, h_j(x)] \leftarrow C[j, h_j(x)] + s_j(x)c.$$

La estimación de cada fila debe deshacer el signo:

$$z_j(x) = s_j(x) C[j, h_j(x)],$$

y la estimación final es

$$\hat{f}(x) = \text{mediana}_{j=1}^d z_j(x).$$

Para estimar frecuencias de paquetes pueden truncar valores negativos a cero al reportar el resultado final.

6. Enunciado

6.1. Actividad 1. Implementación de la ventana deslizante con CMS y CS (2.5 puntos)

Implementen Count-Min Sketch y CountSketch sobre el mismo mecanismo de ventana deslizante.

Requisitos:

1. Implementar un anillo de $m = 6$ subventanas de 10 segundos y un sketch agregado que represente los últimos $W = 60$ segundos.
2. Actualizar el agregado usando explícitamente la linealidad:

$$A \leftarrow A - S_{\text{expira}}$$

al rotar una subventana, y actualizar simultáneamente el sub-sketch actual y A cuando llega un paquete.

3. Usar la misma estructura de ventana para CMS y CS. Sólo deben cambiar las operaciones propias del estimador.
4. Permitir configurar d y w desde la línea de comandos.
5. Mantener también el número exacto de paquetes N_j de la ventana mediante un anillo de seis contadores escalares. Este valor se utilizará para definir el umbral de detección.
6. Validar las estimaciones contra `exact_hh.cpp` para un conjunto de claves y ventanas de la traza sin ataques.

Para la evaluación utilicen $d = 5$ y al menos tres valores de ancho, por ejemplo

$$w \in \{256, 1024, 4096\}.$$

Reporten para CMS y CS el error absoluto y relativo de las frecuencias estimadas y la memoria ocupada por los contadores. No es necesario realizar un barrido exhaustivo de parámetros.

6.2. Actividad 2. Detección de dos ataques (2.5 puntos)

Genere dos trazas a partir de la misma traza base:

1. una con ataque `ddos`;
2. una con ataque `scan`.

Para cada ataque utilicen la clave indicada en el archivo JSON generado por el script `inject_attack.py`:

- para `ddos`, estimar frecuencia por **IP de destino**;
- para `scan`, estimar frecuencia por **IP de origen**.

Utilicen el criterio de heavy hitter definido previamente, con $\phi = 0,01$. Para cada ventana deben comparar la decisión exacta

$$f_j(x) \geq \lceil \phi N_j \rceil$$

obtenida con `exact_hh.cpp`, con la decisión aproximada

$$\hat{f}_j(x) \geq \lceil \phi N_j \rceil$$

obtenida con CMS y CountSketch. El instante de detección corresponde a la primera ventana en que se cumple el criterio para la clave del ataque.

Para cada combinación ataque/sketch/ancho w :

1. Graficar, desde 60 segundos antes del ataque hasta 60 segundos después de su término, la frecuencia exacta y la frecuencia estimada de la clave del ataque. **Superpongan las curvas**: una figura por ataque, con la curva exacta y las de CMS y CS para los tres anchos. Doce figuras separadas no caben en el informe ni permiten comparar.
2. Reportar el error relativo medio (MRE) durante las ventanas afectadas por el ataque. Para evitar divisiones por cero, el promedio debe considerar sólo ventanas en las que la frecuencia exacta sea positiva y la ventana activa contenga tráfico del intervalo de ataque:

$$\text{MRE} = \frac{1}{|J|} \sum_{j \in J} \frac{|\hat{f}_j(x) - f_j(x)|}{f_j(x)},$$

donde J contiene las evaluaciones posteriores al inicio del ataque y anteriores a que todos sus paquetes hayan salido de la ventana, con $f_j(x) > 0$.

Tengan presente que $|J|$ es pequeño, del orden de ocho ventanas, y que el estimador de CountSketch es insesgado: su error es una variable aleatoria de media cero, de modo que el MRE puede salir **no monótono** en w . Eso no es un error de implementación. Reporten junto al promedio el error de cada ventana, o repitan con dos o tres semillas distintas, para distinguir la tendencia del ruido muestral.

3. Medir la **latencia de detección**: diferencia entre el instante de inicio del ataque y la primera evaluación en que la clave satisface el criterio anterior. Como $p = 10$ s, la latencia se observa con resolución de 10 s.

Con los parámetros indicados la latencia será de una o dos evaluaciones, así que no esperen una curva: lo informativo no es el valor en sí, sino si CMS y CS detectan en la *misma* ventana que la referencia exacta. En particular, dado que CMS sobreestima, con w pequeño puede declarar el ataque **una ventana antes** que el conteo exacto. Ese falso positivo es un resultado esperable y deben reportarlo y explicarlo, no corregirlo.

4. Comparar CMS y CS en precisión, memoria y latencia. Expliquen los resultados en términos de colisiones y de la diferencia entre los estimadores mínimo y mediana.

6.3. Estimación del cambio de frecuencia entre ventanas

Además de estimar la frecuencia absoluta de la clave del ataque, estudien el **cambio de frecuencia** entre dos ventanas consecutivas. Definimos

$$\Delta f_j(x) = f_j(x) - f_{j-1}(x).$$

Por lo tanto, $\Delta f_j(x) > 0$ indica que la frecuencia de x aumentó, mientras que $\Delta f_j(x) < 0$ indica que disminuyó.

Por linealidad, para cada sketch puede construirse directamente el arreglo de diferencias

$$\Delta A_j = A_j - A_{j-1},$$

sin volver a procesar los paquetes de ambas ventanas. A diferencia de A_j , ΔA_j representa un vector firmado, ya que $\Delta f_j(x)$ puede ser positivo o negativo.

CountSketch. Utilicen sobre ΔA_j el mismo estimador habitual de CountSketch:

$$\widehat{\Delta f_j}^{CS}(x) = \text{mediana}_{r=1}^d [s_r(x) \Delta A_j[r, h_r(x)]] .$$

El hash de signo permite que CountSketch trate naturalmente incrementos y decrementos de frecuencia.

Count-Min Sketch con estimador de mediana. Para CMS, **no utilicen el estimador mínimo** sobre ΔA_j , porque el vector de diferencias contiene valores negativos y se pierde la propiedad de sobreestimación que justifica el operador mínimo. En esta parte utilicen, con fines experimentales, el estimador

$$\widehat{\Delta f_j}^{CMS-med}(x) = \text{mediana}_{r=1}^d [\Delta A_j[r, h_r(x)]] .$$

Este estimador permite obtener valores positivos y negativos, pero **no corresponde al estimador estándar de Count-Min Sketch y no conserva sus garantías habituales**, ya que las colisiones pueden introducir contribuciones firmadas y sesgadas. Su objetivo en esta tarea es comparar experimentalmente su comportamiento con CountSketch.

Para la clave de cada ataque:

1. calcular el valor exacto $\Delta f_j(x)$ a partir de las frecuencias entregadas por `exact_hh.cpp`;
2. estimar $\Delta f_j(x)$ con CountSketch y con CMS-mediana;
3. graficar $\Delta f_j(x)$, $\widehat{\Delta f_j}^{CS}(x)$ y $\widehat{\Delta f_j}^{CMS-med}(x)$ alrededor del inicio y término del ataque;
4. identificar los mayores incrementos y decrementos y discutir cuál de los dos estimadores aproxima mejor el cambio exacto.

Esta parte debe diferenciarse de la detección por heavy hitters: allí se estima la frecuencia absoluta $f_j(x)$ y CMS utiliza su estimador estándar basado en el mínimo; aquí se estima el cambio firmado $\Delta f_j(x)$ y se compara CountSketch con la variante experimental CMS-mediana.

7. Preguntas obligatorias del informe

El informe debe responder de forma breve las siguientes preguntas:

1. ¿Por qué la linealidad permite mantener la ventana con costo independiente de la cantidad de paquetes que permanecen en ella?
2. ¿Qué diferencias observan entre el error de CMS y CS al reducir w ?

3. ¿Cuál de los dos ataques se detecta con mayor claridad y por qué la definición de la clave es distinta para `ddos` y `scan`?
4. ¿Qué información adicional entrega $\Delta f_j(x) = f_j(x) - f_{j-1}(x)$ respecto de observar sólo la frecuencia de la ventana actual?
5. Compare $\widehat{\Delta f_j}^{CS}(x)$ y $\widehat{\Delta f_j}^{CMS-med}(x)$. ¿Por qué CountSketch puede utilizar su estimador habitual, mientras que en CMS se reemplaza el mínimo por una mediana y se pierden las garantías estándar?

8. Entregables

Cada grupo debe entregar:

- código fuente de CMS y CS con ventana deslizante;
- scripts necesarios para reproducir los experimentos;
- un informe breve, de máximo **6 páginas** sin contar anexos;
- figuras de frecuencia exacta versus estimada para los dos ataques;
- figura de $\Delta f_j(x)$ exacto y de las estimaciones con CountSketch y CMS-mediana;
- una tabla resumen con error, memoria y latencia para CMS y CS;
- una **presentación oral de 10 minutos** por grupo, centrada en el diseño de la ventana deslizante, el uso de la linealidad, los resultados para ambos ataques y las principales conclusiones de la comparación entre CMS y CS.

No deben subir las trazas de MAWI al repositorio. Indiquen la URL de la traza utilizada y la semilla usada para generar los ataques.

9. Rúbrica

Criterio	Puntos
<i>Actividad 1 — implementación</i>	
Implementación correcta de Count-Min Sketch	0.50
Implementación correcta de CountSketch, incluyendo manejo del signo	0.50
Ventana deslizante mediante anillo y suma/resta lineal de sketches	0.75
Alineación correcta de las subventanas, verificada con N_j contra <code>exact_hh</code>	0.25
Validación contra conteo exacto y análisis básico de error/memoria	0.50
<i>Subtotal</i>	<i>2.50</i>
<i>Actividad 2 — detección</i>	
Detección de DDoS con clave y criterio correctos	0.60
Detección de Scan con clave y criterio correctos	0.60
Comparación CMS versus CS: error, memoria y latencia	0.80
Cambio de frecuencia $\Delta f_j(x)$: CountSketch, CMS-mediana y análisis comparativo	0.50
<i>Subtotal</i>	<i>2.50</i>
<i>Transversal</i>	
Discusión, preguntas obligatorias y reproducibilidad	0.50
Presentación oral (10 min): síntesis, claridad, dominio del trabajo y discusión de resultados	0.50
<i>Subtotal</i>	<i>1.00</i>
Total	6.0